MOBILE APP DEVELOPMENT 700

RICHFIELD
richfield.ac.za

Name and Surname: Rahul Sewdutt **Student ITS No:** 402413204

Qualification: B S C I T **Year of Study:** 3rd **Semester:** 2nd

Assignment due date: 31/10/2025 **Date submitted:** 2 9 / 1 0 / 2 0 2 5

QUESTION	EXAMINER MARKS	MODERATOR MARKS	REMARKS

ASSIGNMENT INSTRUCTIONS

Please tick each box to confirm completion.

Use Times New Roman font, size 12, with 1.5 line spacing throughout the document. Apply Harvard Referencing Style for all citations and references.

For essay-style assignments, please include the following sections:

Table of Contents Introduction

Main Body (with relevant subheadings)
Conclusion References

Submit the assignment in PDF format on Moodle. Use the specified cover page provided.

Include a signed declaration of originality.

DECLARATION OF ORIGINALITY:

I hereby declare that this assignment is my own work and has not been copied from any other source except where due acknowledgment is made. I affirm that all sources used have been properly cited and that this submission complies with the institution's policies on academic integrity and plagiarism.

Student Signature: RSewdutt **Date:** 28/10/2026

Table of Contents

1. Introduction 3

1.1 The problem and who the app is for 3

1.2 Aim, scope and summary 3

2. System Design 4

2.1 Architecture and technology 4

2.2 Screen flow 5

2.3 Data model 5

2.4 Database choice and justification 6

2.5 The strict-matching algorithm 7

2.6 Expiry, planner and dashboard design 7

3. Screenshots of Every Output 7

3.1 Pantry management 8

3.2 Suggested recipes and strict matching 10

3.3 Recipe detail and cooking 11

3.4 Weekly meal planner 12

3.5 Settings and dashboard 13

3.6 Persistence, tests and version control 14

4. Key Code Snippets 14

4.1 Normalising ingredient names 14

4.2 The strict-matching rule 15

4.3 Counting days without time-zone errors 16

4.4 Cooking a recipe in one transaction 16

4.5 Upgrading the database without losing data 17

5. Challenges and Solutions 18

5.1 The wrong bottom-navigation tab was highlighted 18

5.2 Upgrading the database without deleting the pantry 18

5.3 Unit conversion bugs found by unit tests 18
6. Conclusion and Reflection 18
6.1 What I learned 18
6.2 What I would improve 19

1. Introduction

1.1 The problem and who the app is for

Food that is bought but never eaten costs households' money and harms the environment. The United Nations Environment Programme estimated that the world wasted about 1.05 billion tonnes of food in 2022, and that households were responsible for 631 million tonnes of it, which is about 60 per cent (UNEP, 2024). The same report puts household waste at roughly 79 kg per person per year (UNEP, 2024). Much of this happens for simple reasons: people forget what they have, lose track of expiry dates, or cannot think of a meal that uses the ingredients that are left, so they buy more food and the old food is thrown away.

Smart Pantry Manager is for anyone who cooks at home and wants to waste less: students sharing a flat, families planning their weekly meals, and busy people who want a quick answer to the question "what can I cook tonight with what I already have?". It needs no account and no internet connection, so it can be used anywhere in the kitchen.

1.2 Aim, scope and summary

The aim was to build a Java Android application that records the ingredients a user has at home and suggests only the recipes they can cook right now, with no shopping trip required. A recipe is suggested only when every ingredient it needs is in the pantry in at least the required quantity. This strict-matching rule is the most important piece of logic in the app.

As the brief requires, the app does not use Google Maps, any mapping SDK or device location services, and it does not process payments or publish to the Play Store.

The app is written in Java in Android Studio and stores its data locally in SQLite. Its features are:

- Pantry management: add, view, edit and delete ingredients with a name, quantity, unit, and an optional expiry date and price.
- A library of 75 recipes, loaded into the database the first time the app runs.
- A Suggested Recipes screen that applies the strict-matching rule, with a separate "Almost There" list for recipes missing exactly one ingredient.
- A Recipe Detail screen with a photo, the ingredient list and the method, and an "I cooked this" button that updates the pantry.

- Expiry highlighting, with a “Use these first” card for food that has expired or is about to expire.
- A weekly meal planner for breakfast, lunch and dinner, which offers recipes that use expiring ingredients first.
- A waste and savings dashboard with monthly charts, and a Settings screen.

The source code is available at: **[GitHub repository link]**.

2. System Design

2.1 Architecture and technology

The code is organised by responsibility so that the decisions the app makes can be tested without an emulator. The activities only handle what the user sees and does. The rules (what to suggest, what is expiring, what a cooked recipe uses) sit in small utility classes with no user-interface code.

Package	Responsibility
activities	One class per screen: Pantry List, Add/Edit Ingredient, Suggested Recipes, Recipe Detail, Meal Planner, Settings and Dashboard.
adapters	RecyclerView adapters that bind pantry items and recipes to list rows.
database	DatabaseHelper (all SQL) and RecipeSeeder (the 75 recipes).
models	Plain data classes: PantryItem, Recipe, RecipeIngredient, MealPlanEntry and FoodEvent.
utils	Application logic: IngredientMatcher, ExpiryUtils, PantryCooker, MealPlanner, WasteStats, InputValidator and BottomNavHelper.
views	BarChartView, a small bar chart drawn on a Canvas, so no chart library is needed.

The brief asks for at least four screens. The app has seven: the Pantry List, Add/Edit Ingredient, Suggested Recipes, Recipe Detail, Settings, and two extra screens, the Meal Planner and the Dashboard.

2.2 Screen flow

Figure 1 shows how the screens connect. The four main screens are reached through a bottom navigation bar (Material Design, n.d.). From the Pantry List, tapping an item or the add button opens the Add/Edit screen through an explicit Intent. Editing passes the item's id as an extra, and adding sends no id. From Suggested Recipes, tapping a recipe opens the Recipe Detail screen with the recipe's id as an extra (Android Developers, n.d.c). The recipe picker, the remove dialog and the cooking confirmation are dialogs on top of their parent screen.

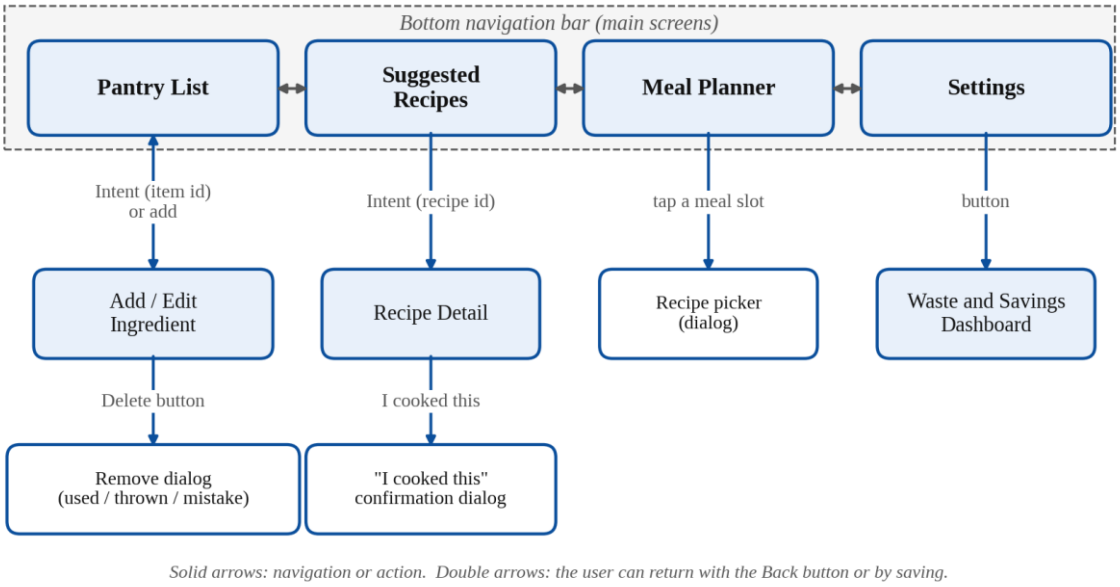


Figure 1: Screen flow of Smart Pantry Manager.

2.3 Data model

Figure 2 shows the five tables in the SQLite database. A recipe has many recipe ingredients, and a meal plan slot refers to one recipe. Pantry items are not linked to recipe ingredients by a foreign key, because the user types pantry names freely. They are matched in code by a normalised name instead.

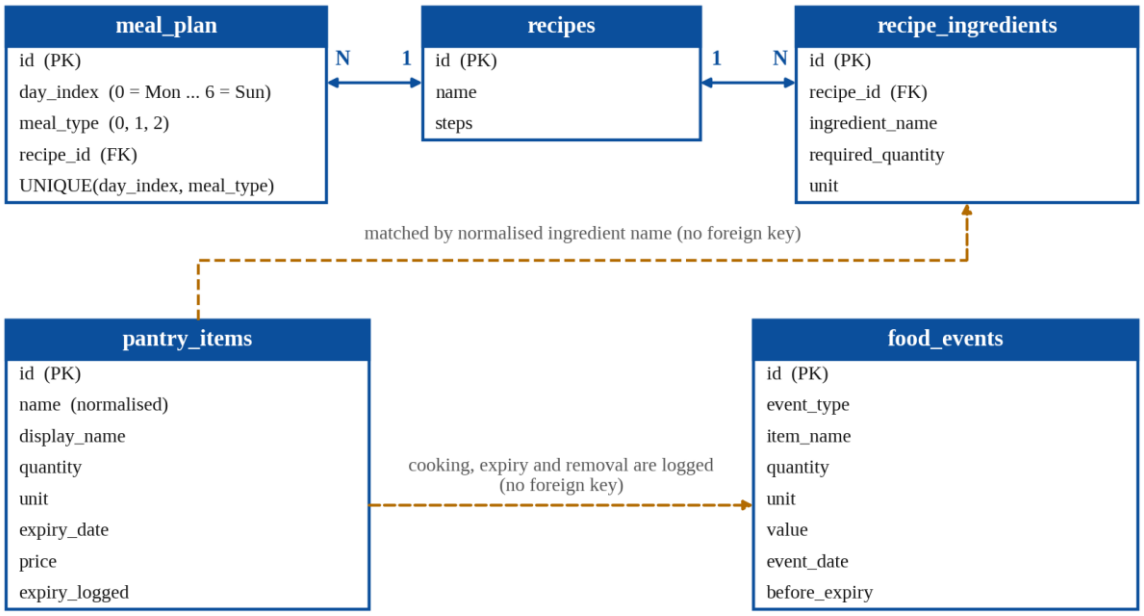


Figure 2: Entity relationship diagram of the database.

- **pantry_items** holds what the user owns: a normalised name for matching, the name as typed, quantity, unit, optional expiry date, optional price, and a flag that records whether its expiry has already been logged.
- **recipes** and **recipe_ingredients** hold the seeded recipes. Each ingredient row stores a required quantity and a unit and points to its recipe.
- **meal_plan** stores one recipe per day and meal. A unique constraint on the day and meal means choosing a new recipe replaces the old one.
- **food_events** is a log of purchases, used items, expired items and discarded items. It is the data behind the dashboard.

The pantry table supports the full create, read, update and delete cycle: Create (Add Ingredient), Read (the pantry list), Update (Edit Ingredient) and Delete (the remove dialog). Because the data is stored in SQLite on the device, it is still there after the app is closed and reopened.

2.4 Database choice and justification

I chose SQLite, accessed through SQLiteOpenHelper (Android Developers, n.d.d), for three reasons. First, it needs no server, account or internet connection, which suits an app that is used in the kitchen. Second, persistence is easy to demonstrate, because the data survives closing and reopening the app. Third, the data is naturally relational: recipes have many ingredients

and planner slots refer to recipes. Firebase and PostgreSQL would have added a network dependency and a backend that the app does not need.

DatabaseHelper creates the tables in onCreate and migrates older versions in onUpgrade. On first launch, RecipeSeeder inserts any of the 75 recipes that are not already in the database.

2.5 The strict-matching algorithm

The matching rule is implemented in IngredientMatcher and works as follows:

1. Each ingredient name, in the pantry and in the recipes, is normalised: it is converted to lower case, extra spaces are removed and simple plurals are made singular, so “tomatoes” matches “tomato”.
2. The pantry is turned into a lookup map from normalised name to item.
3. For every ingredient in a recipe, the pantry item must exist and hold at least the required quantity. Weights are compared in grams (g, kg) and volumes in millilitres (ml, l, tsp, tbsp), so 1 kg of flour satisfies a recipe that needs 500 g. Units that cannot be converted, such as pieces, are compared as plain numbers.
4. If every ingredient passes, the recipe goes in the suggested list. If exactly one fails, it goes in the separate Almost There list. Otherwise it is left out completely.

If the suggested list is empty, the screen shows the message “No recipes match your pantry yet - add more ingredients” instead of a blank screen.

2.6 Expiry, planner and dashboard design

ExpiryUtils calculates the days until an item expires. Items that are past their date are shown in bold red, and items inside the user’s chosen warning window of 1, 3 or 7 days are shown in bold amber. The Meal Planner ranks recipes by how many soon-to-expire pantry items they use, then by how few ingredients are missing. Every cooked recipe, expired item and removed item is written to the food_events table, and the Dashboard totals those events by month.

3. Screenshots of Every Output

This section shows the running app, with a caption for each screen and core function.

3.1 Pantry management

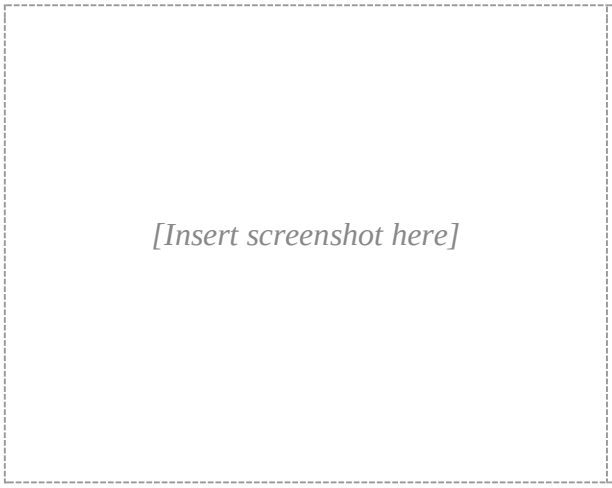


Figure 3: Pantry list with expiry colours and the Use these first card.

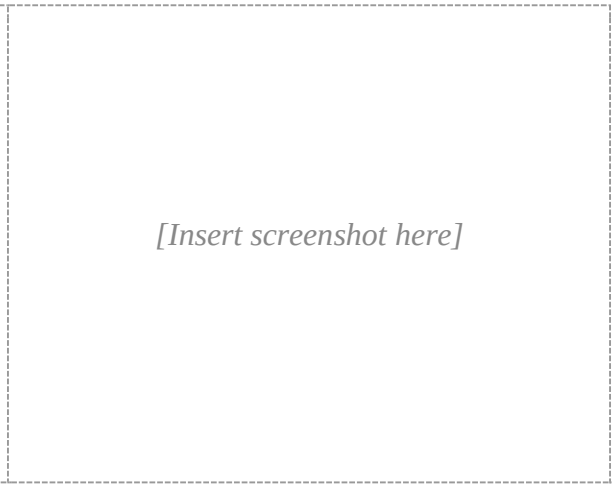


Figure 4: Message shown when the pantry is empty.

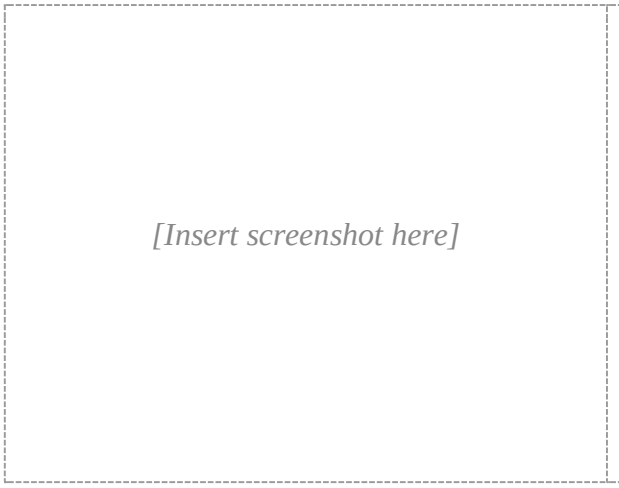


Figure 5: Add Ingredient screen with name, quantity, unit, price and expiry date.

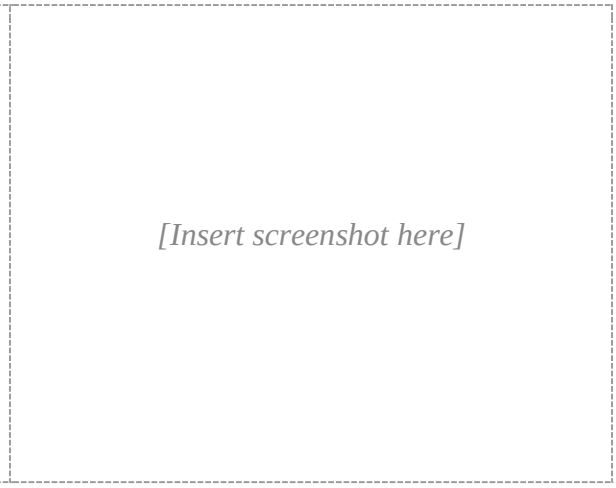


Figure 6: Validation errors for an empty name and an invalid quantity.

[Insert screenshot here]

Figure 7: Edit Ingredient screen with the saved details loaded.

[Insert screenshot here]

Figure 8: Pantry list after the edit, showing the new quantity.

[Insert screenshot here]

Figure 9: Remove dialog: used it up, threw it away, or added by mistake.

[Insert screenshot here]

Figure 10: Pantry list after the item was deleted.

3.2 Suggested recipes and strict matching

[Insert screenshot here]

Figure 11: Suggested Recipes screen listing only fully matched recipes.

[Insert screenshot here]

Figure 12: Almost There list of recipes missing exactly one ingredient.

[Insert screenshot here]

Figure 13: A recipe appears once all of its ingredients are in the pantry.

[Insert screenshot here]

Figure 14: The same recipe disappears after one ingredient is removed.

[Insert screenshot here]

Figure 15: Message shown when no recipes match the pantry.

3.3 Recipe detail and cooking

[Insert screenshot here]

Figure 16: Recipe Detail screen with photo, ingredients and method.

[Insert screenshot here]

Figure 17: Confirmation dialog listing the amounts that will be used.

[Insert screenshot here]

Figure 18: Pantry after cooking, with the used quantities removed.

3.4 Weekly meal planner

[Insert screenshot here]

Figure 19: Weekly planner with breakfast, lunch and dinner slots.

[Insert screenshot here]

Figure 20: Recipe picker with expiring-ingredient recipes at the top.

3.5 Settings and dashboard

[Insert screenshot here]

Figure 21: Settings screen with the alerts toggle, warning window and units.

[Insert screenshot here]

Figure 22: Dashboard summary figures for the current month.

[Insert screenshot here]

Figure 23: Dashboard charts for saved versus wasted and for spending.

[Insert screenshot here]

Figure 24: Dashboard list of recently expired or discarded items.

3.6 Persistence, tests and version control

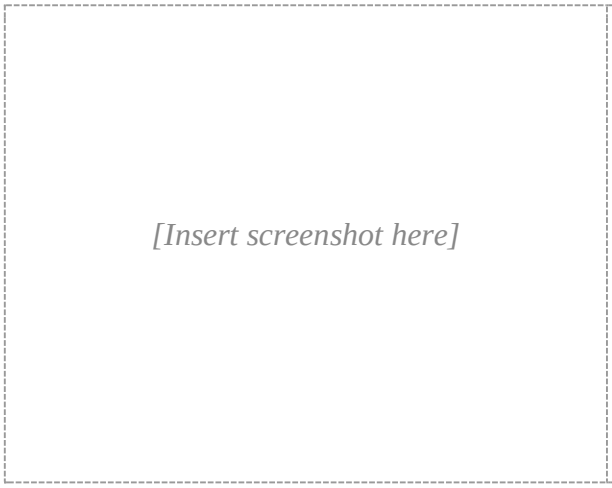


Figure 25: Data still present after closing and reopening the app.

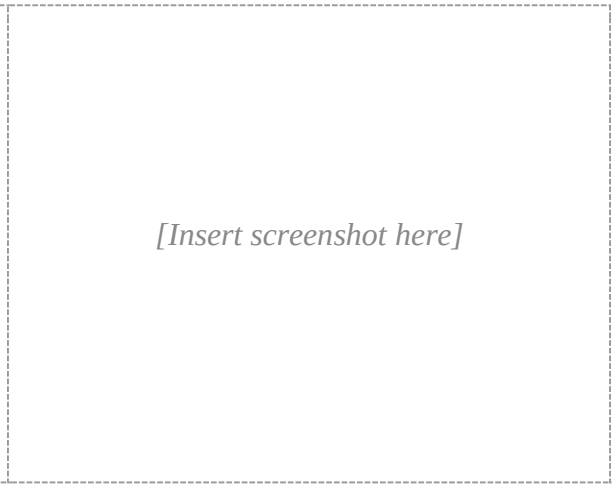


Figure 26: Unit test results in Android Studio, with all tests passing.

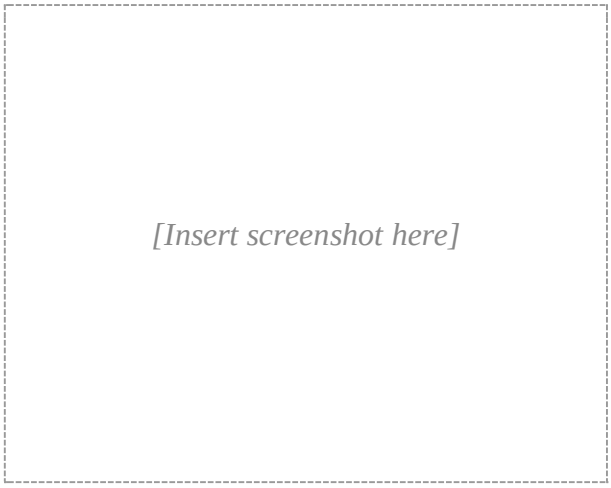


Figure 27: GitHub commit history showing the project's progress.

4. Key Code Snippets

4.1 Normalising ingredient names

```
public static String normalize(String rawName) {
    if (rawName == null) return "";
    String s = rawName.trim().toLowerCase().replaceAll("\\s+", " ");
    if (s.length() <= 3) return s;
    if (s.endsWith("ies")) {
        return s.substring(0, s.length() - 3) + "y";
```

```
    }
    if (s.endsWith("oes") || s.endsWith("ches") || s.endsWith("shes")
            || s.endsWith("sses") || s.endsWith("xes") || s.endsWith("zes")) {
        return s.substring(0, s.length() - 2);
    }
    if (s.endsWith("s") && !s.endsWith("ss") && !s.endsWith("us")) {
        return s.substring(0, s.length() - 1);
    }
    return s;
}
```

This method in PantryItem is what stops the brief's "tomato" versus "tomatoes" problem from breaking the match. It lower-cases the name, collapses spaces and handles the common English plural endings. I kept it to a small set of rules instead of a language library, because the rule only needs to be reliable for ingredient names, and every rule is easy to test.

4.2 The strict-matching rule

```
static boolean isSatisfied(RecipeIngredient required, Map<String, PantryItem>
pantryByName) {
    PantryItem pantryItem = pantryByName.get(required.getIngredientName());
    if (pantryItem == null) return false;

    Double reqQty = toBaseUnit(required.getRequiredQuantity(), required.getUnit());
    Double haveQty = toBaseUnit(pantryItem.getQuantity(), pantryItem.getUnit());

    if (reqQty != null && haveQty != null) {
        return haveQty >= reqQty;
    }
    // Units not comparable (e.g. "pcs") or unspecified: fall back to raw quantity
comparison.
    return pantryItem.getQuantity() >= required.getRequiredQuantity();
}
```

This is the core of the assignment. An ingredient only passes if it exists in the pantry and the quantity is large enough after both amounts are converted to a common base unit. A recipe is suggested only if this returns true for every one of its ingredients, so a recipe with one missing

ingredient can never reach the main list. The same method is used when cooking and when ranking planner recipes, so the app cannot suggest something it cannot cook.

4.3 Counting days without time-zone errors

```
public static Integer daysUntil(String isoExpiry, int todayYear, int todayMonth, int
todayDay) {
    Long expiry = parseToDayNumber(isoExpiry);
    if (expiry == null || isInvalidDate(todayYear, todayMonth, todayDay)) return null;
    return (int) (expiry - toDayNumber(todayYear, todayMonth, todayDay));
}

private static long toDayNumber(int y, int m, int d) {
    long yy = m <= 2 ? y - 1 : y;
    long era = Math.floorDiv(yy, 400);
    long yoe = yy - era * 400;
    long doy = (153L * (m + (m > 2 ? -3 : 9)) + 2) / 5 + d - 1;
    long doe = yoe * 365 + yoe / 4 - yoe / 100 + doy;
    return era * 146097 + doe - 719468;
}
```

Subtracting two timestamps gives the wrong number of days on a day that is 23 or 25 hours long because of daylight saving. Instead, each date is converted to a day number from its year, month and day only, and the two numbers are subtracted, so no time zone is involved. Invalid dates such as 29 February in a non-leap year return null. Unit tests check this in several time zones.

4.4 Cooking a recipe in one transaction

```
public void applyCook(List<PantryCooker.Change> changes, List<FoodEvent> events) {
    SQLiteDatabase db = getWritableDatabase();
    db.beginTransaction();
    try {
        for (PantryCooker.Change c : changes) {
            String[] idArg = new String[]{String.valueOf(c.item.getId())};
            if (c.delete) {
                db.delete(TABLE_PANTRY, COL_PANTRY_ID + "=?", idArg);
```

```
            } else {
                ContentValues values = new ContentValues();
                values.put(COL_PANTRY_QUANTITY, c.newQuantity);
                values.put(COL_PANTRY_PRICE, WasteStats.priceAfterUse(c.item,
c.newQuantity));
                db.update(TABLE_PANTRY, values, COL_PANTRY_ID + "=?", idArg);
            }
        }
        for (FoodEvent e : events) insertEvent(db, e);
        db.setTransactionSuccessful();
    } finally {
        db.endTransaction();
        db.close();
    }
}
```

When the user cooks a recipe, several pantry rows change and several log rows are added. Wrapping them in one transaction means they either all succeed or none do, so the pantry and the dashboard can never disagree if something fails part-way. Items that are used up are deleted, and partly used items keep a proportional share of their price.

4.5 Upgrading the database without losing data

```
public void onUpgrade(SQLiteDatabase db, int oldVersion, int newVersion) {
    // Keep the user's pantry: only add what is new.
    if (oldVersion < 2) {
        createMealPlanTable(db);
    }
    if (oldVersion < 3) {
        db.execSQL("ALTER TABLE " + TABLE_PANTRY + " ADD COLUMN " +
COL_PANTRY_PRICE + " REAL");
        db.execSQL("ALTER TABLE " + TABLE_PANTRY + " ADD COLUMN " +
COL_PANTRY_EXPIRY_LOGGED
                + " INTEGER NOT NULL DEFAULT 0");
        createFoodEventsTable(db);
    }
```

```
}
```

Each schema version adds only what is new, using CREATE TABLE and ALTER TABLE ADD COLUMN (SQLite, n.d.). Because each step is guarded by the old version number, a database at any earlier version is brought up to date in order, and the user's pantry is kept.

5. Challenges and Solutions

5.1 The wrong bottom-navigation tab was highlighted

After switching screens, the bottom bar sometimes highlighted the wrong tab. For example, tapping Suggestions and then Pantry left Suggestions highlighted on the Pantry screen. The cause was that the navigation listener returned true, which marks the tapped tab as selected, so the screen being left changed its own highlight before the new screen opened. When the user later returned to that screen, it still showed the wrong tab. I fixed it by returning false from the listener whenever a tap starts a navigation, so each screen keeps its own tab highlighted.

5.2 Upgrading the database without deleting the pantry

The generated onUpgrade method dropped every table and recreated it. That is fine while developing but would delete a real user's pantry whenever I added a table or column. I replaced it with the version-guarded migration in section 4.5, so older databases only gain the new tables and columns. I then checked that existing pantry items were still present after installing the new version over the old one.

5.3 Unit conversion bugs found by unit tests

I wrote JUnit tests that simulate cooking every one of the 75 recipes (Android Developers, n.d.a; JUnit, n.d.). They showed that teaspoons and tablespoons were not converted: a pantry with 1 tablespoon of oil did not satisfy a recipe that needed 3 teaspoons, and cooking would have subtracted the wrong amount. I added the conversions (1 tsp = 5 ml, 1 tbsp = 15 ml) to the matcher, and all the tests then passed. Testing the logic separately from the screens made this kind of bug easy to find.

6. Conclusion and Reflection

6.1 What I learned

The project covered the main topics of the module in one application. I used the Activity lifecycle (Android Developers, n.d.e) to refresh lists and statistics in onResume, so a screen

always shows current data when the user returns to it. I used Intents to move between screens and pass ids, RecyclerView with custom adapters to show database records (Android Developers, n.d.b), and SQLiteOpenHelper for persistent storage with a full create, read, update and delete cycle.

The most useful lesson was to keep the decisions out of the activities. Putting the matching, expiry and cooking rules in plain Java classes meant I could test them quickly, and the tests found real mistakes that I would probably have missed by tapping through the app. I also learned that database changes need a migration plan from the start, and that a feature is only finished when its data is saved, shown and checked.

6.2 What I would improve

- Add Android notifications so the user is warned about expiring food even when the app is closed. At the moment the warnings appear inside the app only.
- Build a shopping-list screen for the meal planner. The calculation of what is missing is written and tested, but it has no screen yet.
- Add search and sorting to the pantry list, and let users add their own recipes, which would give full create, read, update and delete on a second table.
- Apply the metric or imperial units setting to the quantities that are displayed, and combine duplicate pantry rows for the same ingredient.
- Move the database code to the Room library to get compile-time checking of queries.